CAMPUSCONNECT

COLLEGE EVENT MANAGEMENT SYSTEM

MICRO-PROJECT REPORT

Project Type	Java desktop application
Interface	Java Swing GUI
Primary Users	Students and event organizers
Storage	In-memory collections (current version)

Prepared for academic submission

Project source reviewed: campusconnect.java

1. Introduction

CampusConnect is a Java-based desktop application designed to support basic college event management. Colleges regularly conduct technical festivals, workshops, seminars, cultural programmes, competitions and other student activities. When information about these events is shared through separate messages, notices or informal lists, students may find it difficult to discover events and organizers may find it difficult to maintain participant details.

The project provides a single graphical interface through which students can explore listed events, view event information and submit a registration. Organizers can create events, review participant entries and view a summary report. The interface is built with Java Swing and changes between pages according to the selected action and user role.

CampusConnect is implemented as a compact academic prototype. The current program stores event names and participant entries in ArrayList collections and keeps account and event details in application variables. It does not connect to a database or remote server, so the information is temporary and is cleared when the application closes.

1.1 Objectives

- Provide a simple interface for students and organizers.
- Allow students to browse available events and see event details.
- Collect a student name for event registration and display registrations.
- Allow organizers to create events and view participants.
- Generate a basic summary containing event and registration counts.
- Demonstrate object-oriented programming concepts and event-driven GUI development in Java.

1.2 Intended Users

The main users are students who want to find and register for campus activities, and organizers who need a basic way to publish events and view sign-ups. The application begins with a role-selection screen and presents different dashboards for the two roles.

2. Scope of the Micro-Project

The scope of CampusConnect is limited to the essential workflow of a college event portal: entering the application, choosing a role, navigating to a dashboard, exploring or creating events, registering participants and viewing a simple report. This scope is suitable for a micro-project because it demonstrates a complete user journey without requiring a server or complex deployment.

2.1 Features Included

- **Role selection:** the user chooses Student or Organizer.
- **Login screen:** a username and password form is shown. In the current implementation, the login button does not validate credentials; it uses the entered username or a demo label to open the selected dashboard.
- **Student registration form:** collects name, roll number, department, semester, college, email, phone, username and password, with basic empty-field checks.
- **Student dashboard:** provides access to events, registrations, profile and logout.
- **Event browsing and details:** lists event names and displays the current event description, date and venue.
- **Event sign-up:** records a participant entry associated with an event name.
- **Organizer tools:** create events, view participant entries and view a summary report.

2.2 Boundaries and Limitations

- Data is held in memory; there is no database, file persistence or cloud storage.
- The prototype supports one current event-detail set (name, description, date and venue) while the event list stores names. Multiple event records are not modeled as separate objects.
- Login is a navigation demonstration rather than secure authentication.
- There are no role permissions enforced beyond the interface flow, and no organizer account-registration workflow.
- The report displays simple counts and current event details, not analytics or downloadable reports.

2.3 Possible Extensions

Future versions could introduce a database, password hashing, validated login, separate Event and Student objects, unique registration IDs, event-specific participant lists, duplicate-registration checks, search and filtering, capacity limits, email notifications and report export.

3. Technologies Used

The source file imports Java Swing, AWT and ArrayList. These standard Java libraries are sufficient to build the current graphical prototype without third-party dependencies.

Technology	Use in the project	Examples from source
Java	Main programming language; class-based application logic.	campusconnect class, fields, methods, conditions
Java Swing	Desktop GUI components and dialog boxes.	JFrame, JPanel, JLabel, JButton, JTextField, JOptionPane
AWT	Layouts, colours, dimensions, fonts and cursor behaviour.	BorderLayout, GridLayout, Color, Font, Dimension
ArrayList	Stores event names and participant entries while the app is running.	eventList, participants
Lambda expressions / Runnable	Connects button actions to navigation or operations.	addButton(..., Runnable action)
Swing event listeners	Responds to user actions such as button clicks.	addActionListener

3.1 GUI Layout and Styling

The main window is a JFrame titled “CampusConnect - College Event Portal”. It uses a BorderLayout as the main container. Pages are constructed with JPanel objects, a header, a centered card and a footer. BoxLayout arranges controls vertically, GridLayout arranges department checkboxes in a grid, and JScrollPane makes longer pages scrollable. Colours, borders, spacing and fonts are set directly in Java.

3.2 Runtime and Execution

A Java Development Kit (JDK) and a Java IDE or command-line compiler are sufficient to compile and run the program. Since the program uses standard library classes, no external library installation is required. The main method starts the GUI through SwingUtilities.invokeLater, which schedules GUI creation on Swing’s event-dispatch thread.

4. Problem Statement

College event information and registrations can become fragmented when event announcements, student sign-ups and organizer records are maintained separately. Students need a convenient way to discover activities and understand the date, venue and description before registering. Organizers need a simple overview of the events created and the students who have signed up.

Problem statement: To develop a simple desktop-based college event management application that provides a common interface for students to browse and register for campus events and for organizers to create events, view participants and review a basic event summary.

4.1 Functional Requirements

- The application shall provide Student and Organizer role choices.
- The application shall display a role-specific dashboard.
- Students shall be able to view the event list and event details.
- Students shall be able to submit a registration name for an event.
- Students shall be able to view matching registration entries.
- Organizers shall be able to create an event by entering its name, date, venue and description.
- Organizers shall be able to view the participant list and summary report.

4.2 Non-Functional Requirements

- **Usability:** labels and buttons should make the workflow easy to follow.
- **Responsiveness:** screen changes should happen immediately after user actions.
- **Maintainability:** repeated GUI construction is placed in helper methods.
- **Portability:** the program uses standard Java desktop libraries and can run on a compatible system with a JDK/JRE.
- **Data integrity:** basic empty-field validation is included, although stronger validation is needed for production use.

4.3 Proposed Solution

CampusConnect addresses the stated problem through a role-based graphical workflow. The student side focuses on event discovery and sign-up, while the organizer side focuses on event creation and participant review. The design is intentionally lightweight and appropriate for demonstrating a working concept in a micro-project setting.

5. Literature Review / Background

This section provides the conceptual background relevant to the submitted implementation. The source file itself does not contain citations to published studies or a formal literature survey; therefore, the discussion below is a technology and design background rather than a claim that specific papers were reviewed.

5.1 Event Management Systems

An event management system organizes information about events and supports tasks such as publishing details, collecting registrations and reviewing attendance or participant lists. A college-focused system commonly needs event titles, descriptions, dates, venues and a relationship between each event and its registered participants. CampusConnect implements a basic subset of these tasks through its event list and participant list.

5.2 Desktop GUI Applications

A graphical user interface lets people interact with a program through visual controls rather than text commands. Java Swing provides desktop components such as frames, panels, labels, text fields, buttons, checkboxes and combo boxes. The project uses these components to create forms, dashboards and informational screens. Layout managers help arrange components and reduce the need for fixed pixel positioning.

5.3 Event-Driven Programming

In event-driven programming, the application responds to actions such as button clicks. CampusConnect attaches listeners to buttons and uses lambda expressions to invoke methods such as showEvents, showCreateEvent and showParticipants. This separates the user action from the page-building method and makes navigation straightforward for a small application.

5.4 In-Memory Data Structures

ArrayList is a resizable collection suitable for maintaining a sequence of values during execution. In this project, eventList stores event names and participants stores strings combining a participant name and event name. This is simple for a demonstration, but it does not provide durable storage, relational integrity or structured event-specific records.

5.5 Design Implications

The current design favors clarity and a small implementation footprint. A production system would normally separate the interface, business logic and data-access layer; model users, events and registrations as distinct entities; persist information in a database; and validate identity and permissions. These are recommended future improvements, not features claimed for the current code.

6. Implementation

The application is implemented as one Java class with methods that build each screen and handle user actions. Navigation is performed by replacing the content of the main panel. Shared helper methods create the common page layout, header, card, fields and buttons.

6.1 Application Startup

The constructor sets the window title, dimensions, minimum size and close operation. It creates the main BorderLayout panel, inserts the default event name “Tech Fest 2026” into the event list, opens the role-selection screen and makes the frame visible. The main method launches object creation using SwingUtilities.invokeLater.

6.2 Student Workflow

- The user selects Student and opens the login screen.
- The login screen accepts a username; the current implementation does not authenticate the password.
- The student dashboard provides Explore Events, My Registrations, My Profile and Logout.
- The event list opens event details showing description, date and venue.
- The registration form checks that the name is not empty, then adds a participant string if it is not already present.
- The profile screen displays saved registration-form fields, using “Not provided” for empty values.

6.3 Organizer Workflow

- The user selects Organizer and opens the organizer dashboard.
- Create New Event collects the event name, date, venue and description and rejects empty fields.
- A new event name is appended to eventList if it is not already present.
- View Participants displays all participant strings.
- View Reports displays event count, registration count and the current event’s name, date and venue.

6.4 Validation and User Feedback

JOptionPane displays messages for missing registration or event details and confirms successful actions. The signup form checks required fields and requires a department selection. These checks reduce incomplete submissions but do not replace full validation for email, phone number, unique account names, event dates or secure authentication.

6.5 Data Handling

The application stores values in instance fields and ArrayList collections. Because there is no file or database layer, records remain available only while the application process is running. The prototype should be described as an in-memory application rather than a persistent registration platform.

7. Details of Java Classes and Architecture

7.1 Main Class

campusconnect extends JFrame. It is the sole application class in the submitted source file. It contains the main window, application state, UI helper methods, screen-navigation methods and event-handling logic. The class name is lowercase in the source; Java convention would normally use CampusConnect, provided the filename is updated accordingly.

7.2 Important Fields

Field / group	Purpose
main	Main JPanel used to replace the current screen.
role, username	Track the selected role and current display username.
registeredName, registeredRoll, registeredDepartment, registeredSemester, registeredCollege, registeredEmail, registeredPhone, registeredUsername, registeredPassword	Hold the most recently submitted student registration details in memory.
eventName, eventDescription, eventDate, eventVenue	Store the current event's details for the event-detail and report screens.
eventList	ArrayList of event names.
participants	ArrayList of participant/event strings.
loginUsername, loginPassword	References to the login form controls.

7.3 Important Methods

- **showRoleSelection(), showLogin(), showSignup():** build role selection, login and student sign-up screens.
- **showStudentDashboard(), showOrganizerDashboard():** create role-specific dashboards.
- **showEvents(), showEventDetails(), showRegister():** implement event discovery and registration.
- **showMyRegistrations(), showProfile():** display student registration and profile information.
- **showCreateEvent(), showParticipants(), showReport():** provide organizer operations.
- **createOuterPage(), addHeader(), createCard(), createButton(), addButton(), addField(), displayCard(), showPage():** reusable interface helpers.

7.4 Architecture

The current architecture is a single-class, event-driven desktop design. The presentation layer (Swing components), application logic (validation and navigation) and in-memory data state coexist in campusconnect. A future layered architecture could split these responsibilities into UI screens, service classes (EventService and RegistrationService), model classes (Student, Event and Registration) and a repository/database layer.

8. Results & Discussion

The source code implements the principal screens and actions expected from a small college event management prototype. When run in a compatible Java environment, the program opens a role-selection screen and allows navigation through the student and organizer workflows. The results described here are based on inspection of the submitted source; no separate runtime test log or measured user evaluation was provided.

8.1 Functional Results

Feature	Implemented behaviour	Observation
Role selection	Student / Organizer buttons	Role determines dashboard and some visible actions.
Student signup	Collects student details and checks required fields	Details are stored in memory only.
Event browsing	Lists event names and displays event details	Detailed data is held for the current event state.
Event registration	Adds a name-event string to participant list	Duplicate identical strings are avoided.
Event creation	Accepts name, date, venue and description	Only event names are stored in the list.
Participant/report views	Shows participant entries and simple counts	Reports are basic summaries, not analytics.

8.2 Discussion and Limitations

The prototype demonstrates screen navigation, form handling, button events and collection-based data handling. Reusable helper methods reduce repeated layout code. However, the login flow does not validate credentials, the password is stored as a plain String after signup, and no database is used. The event list and current event-detail fields are not modeled as separate event records, so multiple events may not retain independent descriptions, dates and venues. Participant records are strings rather than structured registration objects.

8.3 Suggested Testing

- Check that each role button opens the correct login/dashboard path.
- Submit blank signup and event forms and confirm validation messages.
- Create two event names and inspect the event list and report.
- Register a student and confirm the participant appears in the organizer list.
- Close and restart the program to verify that in-memory data is not persistent.

9. Conclusion

CampusConnect presents a basic Java Swing solution for college event discovery, registration and organizer review. It demonstrates the use of a JFrame-based window, Swing components, layout managers, event listeners, lambda expressions, conditional logic and ArrayList collections. The project provides a foundation for learning GUI-based application development and translating a practical campus need into software features.

For further development, the application should introduce structured model classes for students, events and registrations; persistent storage through a database; secure password handling and actual authentication; stronger input validation; and event-specific participant management. With these improvements, CampusConnect could evolve from a classroom prototype into a more reliable college event management system.

Source basis: campusconnect.java, submitted by the project author. The report describes the current implementation and identifies proposed enhancements separately.